

Files

Reading from files

Writing to files

Reading for formatted files

Writing to formatted files

Files

Trade offs

	Sequential files	Random Files
example:	text file	database
space:	smaller	larger
speed: (time)	slower	faster

The space versus time tradeoff is a normal tradeoff in computing and media.

For example: cassette tape versus mp4 player

Reading from Writing to file

```
namespace FileExample;

/*
    Notice that the output file has an extra newline
    at the end.
*/

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("File Example\n");
        try
        {
            StreamReader reader = new StreamReader("../text.txt");
            StreamWriter writer = new StreamWriter("../OutputFile.txt");
            while (!reader.EndOfStream)
            {
                String line = reader.ReadLine();
                Console.WriteLine(line);
                writer.WriteLine(line);
            }
            writer.Close();
            reader.Close();
        }
        catch(Exception e)
        {
            Console.WriteLine(e);
        }
    }
}
```

Formatted Data

Data is often formatted. (You often to that to make it easy to access and read.)

Sometimes the data is comma delimited (separated). You often find .csv files on the internet. CSV stands for Comma Separated Values.

Just like when reading data from the user, you need to make sure that your data is the correct type when you read from a file. So you need to know what type(s) of data you are reading and put the data into the correct spots. Often you will need use Lists to do this.

Formatted Data

```
namespace FileDelimitedExample;
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Formatted Data Example!");
        try
        {
            StreamReader reader = new StreamReader("../../CommaDelimited.txt");
            StreamWriter writer = new StreamWriter("../../TabDelimited.txt");

            while (!reader.EndOfStream)
            {
                String line = reader.ReadLine();
                Console.WriteLine(line);
                String [] data = line.Split(',');
                String name = data[0];
                char ch = Convert.ToChar(data[1]);
                int i = Convert.ToInt32(data[2]);
                writer.WriteLine($"{name}\t{ch}\t{i}");
                //writer.WriteLine(name + '\t' + ch + '\t' + i);
            }
            writer.Close();
            reader.Close();
        }
        catch(Exception e)
        {
            Console.WriteLine(e);
        }
    }
}
/*
    Notice that the output file has an extra newline
    at the end.
*/
```

Formatted Data

Abraham,t,13

Oscar,P,25

Pablo,Q,99

Michael,T,155

Abraham t 13

Oscar P 25

Pablo Q 99

Michael T 155